Autonomous Vehicle – Fuzzy Inference System

```
# Fuzzy Inference System for an Autonomous Vehicle
# Input : Distance from obstacle (metres)
# Output: Vehicle speed (km/h)
# Mamdani inference with centroid defuzzification.

def triangular(x, a, b, c):
    if x < a or x > c:
        return 0.0
    if x == b:
        return 1.0
    if x < b:
        return (x - a) / (b - a) if b != a else 1.0
    return (c - x) / (c - b) if c != b else 1.0


def obstacle_memberships(distance):
    return {
        "near": triangular(distance, 0, 0, 30),
        "medium": triangular(distance, 10, 40, 70),
        "far": triangular(distance, 50, 100, 100)
    }


def speed_memberships(speed):
    return {
        "slow": triangular(speed, 0, 0, 30),
        "moderate": triangular(speed, 10, 40, 70),
        "fast": triangular(speed, 50, 100, 100)
    }


# Fuzzy rules:
# IF obstacle is near THEN speed is slow.
# IF obstacle is medium THEN speed is moderate.
# IF obstacle is far THEN speed is fast.

def fuzzy_speed_control(distance):
    if not 0 <= distance <= 100:
        raise ValueError("Distance must be between 0 and 100 metres.")

    obstacle = obstacle_memberships(distance)

    # Rule firing strengths (Mamdani minimum implication).
    rules = [
        (obstacle["near"], "slow"),
        (obstacle["medium"], "moderate"),
        (obstacle["far"], "fast")
    ]

    numerator = 0.0
    denominator = 0.0
    step = 0.5

    # Aggregate output membership using maximum operator,
    # then calculate centroid (centre of area).
    speed = 0.0
    while speed <= 100:
        aggregated = 0.0

        for strength, output_set in rules:
            membership = speed_memberships(speed)[output_set]
            clipped = min(strength, membership)
            aggregated = max(aggregated, clipped)

        numerator += speed * aggregated * step
        denominator += aggregated * step
        speed += step

    if denominator == 0:
        return 0.0

    return numerator / denominator


try:
    distance = float(input(
        "Enter obstacle distance in metres (0-100): "
    ))

    recommended_speed = fuzzy_speed_control(distance)
```

```
    print("Obstacle distance:", distance, "metres")
    print("Recommended vehicle speed:",
          round(recommended_speed, 2), "km/h")

    if distance <= 10:
        print("Action: Apply brakes and stop safely.")
    elif distance <= 40:
        print("Action: Reduce speed and proceed cautiously.")
    else:
        print("Action: Safe distance detected; normal speed.")

except ValueError as error:
    print("Invalid input:", error)
```